DATA SCIENCE • MODULE 05

Feature Engineering

Build Better Inputs → Get Better Model Outputs

■■ FEATURE CREATION & SELECTION

■ LEARNING OUTCOME

By the end of this PDF you will create, transform, encode, and select features that maximize model performance — the single highest-leverage skill in applied data science.

■ Skills You Will Gain
• Create new features from existing columns
• Encode categorical variables (label, one-hot, target encoding)
• Scale and normalize numeric features
• Handle high-cardinality and rare categories
• Apply feature selection: filter, wrapper, embedded methods
• Reduce dimensionality with PCA

■ **Better features beat better algorithms. A simple logistic regression with great features often outperforms a complex neural network with raw features.**

SECTION 1

Creating New Features

```
# Arithmetic interactions df['Revenue_per_Order'] = df['TotalRevenue'] / df['OrderCount']
df['Profit_Margin'] = (df['Revenue']-df['Cost']) / df['Revenue'] * 100 # Ratios and rates
df['Click_Rate'] = df['Clicks'] / df['Impressions'] df['Churn_Risk'] = df['Complaints'] /
df['Tenure_Months'] # Date-derived features df['Account_Age_Days'] =
(pd.Timestamp.today()-df['Signup_Date']).dt.days df['Is_Weekend'] =
df['Date'].dt.dayofweek.isin([5,6]).astype(int) df['Month_Sin'] = np.sin(2*np.pi*df['Month']/12) #
cyclical encoding df['Month_Cos'] = np.cos(2*np.pi*df['Month']/12) # Binning / bucketing
df['Age_Group'] = pd.cut(df['Age'], bins=[0,18,35,55,100], labels=['Teen','Young Adult','Mid
Adult','Senior']) df['Income_Quartile'] = pd.qcut(df['Income'], q=4, labels=['Q1','Q2','Q3','Q4'])
```

SECTION 2

Encoding Categorical Variables

Method	When to Use	Code
Label Encoding	Ordinal categories (Low<Med<High)	LabelEncoder().fit_transform(col)
One-Hot Encoding	Nominal, <10 categories	pd.get_dummies(df,columns=['col'])
Target Encoding	High cardinality (City, ZIP code)	df.groupby('City')['target'].mean()
Frequency Encoding	Count of each category	df['col'].map(df['col'].value_counts())
Binary Encoding	Many categories, save memory	category_encoders BinaryEncoder
Ordinal Encoding	Ordered categories with custom order	OrdinalEncoder(categories=[order_list])

```
# Target Encoding (mean of target per category) from category_encoders import TargetEncoder te =
TargetEncoder() X_train['City_enc'] = te.fit_transform(X_train['City'], y_train)
X_test['City_enc'] = te.transform(X_test['City'])
```

SECTION 3

Feature Scaling

	Formula	When to Use
	(x - mean) / std → N(0,1)	Linear models, SVM, neural nets, PCA
	(x - min) / (max - min) → [0,1]	Neural nets, KNN, when need bounded range
	(x - Q2) / IQR	When data has outliers — uses median
	x / max(|x|) → [-1,1]	Sparse data (TF-IDF text features)
	log(x+1)	Right-skewed: price, income, count
	Optimized power transform	Any positive numeric feature

```
from sklearn.preprocessing import StandardScaler, RobustScaler scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # fit on train ONLY X_test_scaled =
scaler.transform(X_test) # transform test with train params # CRITICAL: Always fit scaler on
training data only! # Fitting on test data causes data leakage.
```

SECTION 4

Feature Selection

1	**Filter Methods** Correlation, chi-square, ANOVA F-score — score each feature independently
2	**Wrapper Methods** RFE (Recursive Feature Elimination) — iteratively removes weakest features
3	**Embedded Methods** LASSO (L1 regularization) and tree-based feature importance
4	**Variance Threshold** Remove near-constant features: VarianceThreshold(threshold=0.01)
5	**Mutual Information** Non-linear relationship detection: mutual_info_classif()

```
from sklearn.feature_selection import SelectKBest, chi2, RFE, mutual_info_classif from
sklearn.linear_model import LassoCV from sklearn.ensemble import RandomForestClassifier # Filter:
Select top 10 features by chi2 selector = SelectKBest(chi2, k=10) X_new =
selector.fit_transform(X, y) # Wrapper: RFE with Random Forest rfc =
RandomForestClassifier(n_estimators=100) rfe = RFE(estimator=rfc, n_features_to_select=15)
rfe.fit(X_train, y_train) selected = X.columns[rfe.support_] # Embedded: feature importance from
tree model rfc.fit(X_train, y_train) importances = pd.Series(rfc.feature_importances_,
index=X.columns) importances.sort_values(ascending=False).head(15).plot(kind='barh')
```

SECTION 5

Dimensionality Reduction with PCA

```
from sklearn.decomposition import PCA # Reduce to 2D for visualization pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled) # Choose components explaining 95% variance pca_full = PCA()
pca_full.fit(X_scaled) cumvar = pca_full.explained_variance_ratio_.cumsum() n_components =
np.argmax(cumvar >= 0.95) + 1 print(f'{n_components} components explain 95% of variance') #
Visualize explained variance plt.plot(cumvar) plt.axhline(0.95, color='red', linestyle='--')
plt.xlabel('Number of Components') plt.ylabel('Cumulative Explained Variance')
```

■ PRACTICE QUESTIONS

Q1. Why should you fit a scaler only on training data, not test data?

> ***Answer:*** *Fitting on test data leaks information about the test set into training — artificially inflating performance. Always fit on train, transform both.*

Q2. When would you use target encoding instead of one-hot encoding?

> ***Answer:*** *When a categorical column has very high cardinality (hundreds of cities or ZIP codes) — one-hot creates too many columns and causes the curse of dimensionality.*

Q3. What does cyclical encoding of month achieve?

Answer: *It preserves the circular nature of time: December (12) should be close to January (1), not far away. sin/cos encoding captures this proximity.*

Q4. What is the curse of dimensionality?

Answer: *As number of features grows, data becomes increasingly sparse. Models need exponentially more data to generalize. Feature selection and PCA combat this.*

Q5. What does LASSO do to irrelevant features?

Answer: *LASSO (L1 regularization) shrinks irrelevant feature coefficients exactly to zero, effectively performing automatic feature selection.*

■■ Feature Engineering Cheat Sheet	
`pd.cut(col, bins, labels)`	Bin numeric to categories
`pd.qcut(col, q)`	Quantile-based binning
`pd.get_dummies(df,columns)`	One-hot encoding
`LabelEncoder().fit_transform`	Label encoding
`TargetEncoder().fit_transform`	Target encoding
`StandardScaler()`	Standardize to N(0,1)
`RobustScaler()`	Outlier-resistant scaling
`np.log1p(col)`	Log transform skewed
`SelectKBest(chi2,k=10)`	Top-k feature selection
`RFE(estimator,n_features)`	Recursive elimination
`RandomForest.feature_importances_`	Tree-based importance
`PCA(n_components=2)`	Dimensionality reduction